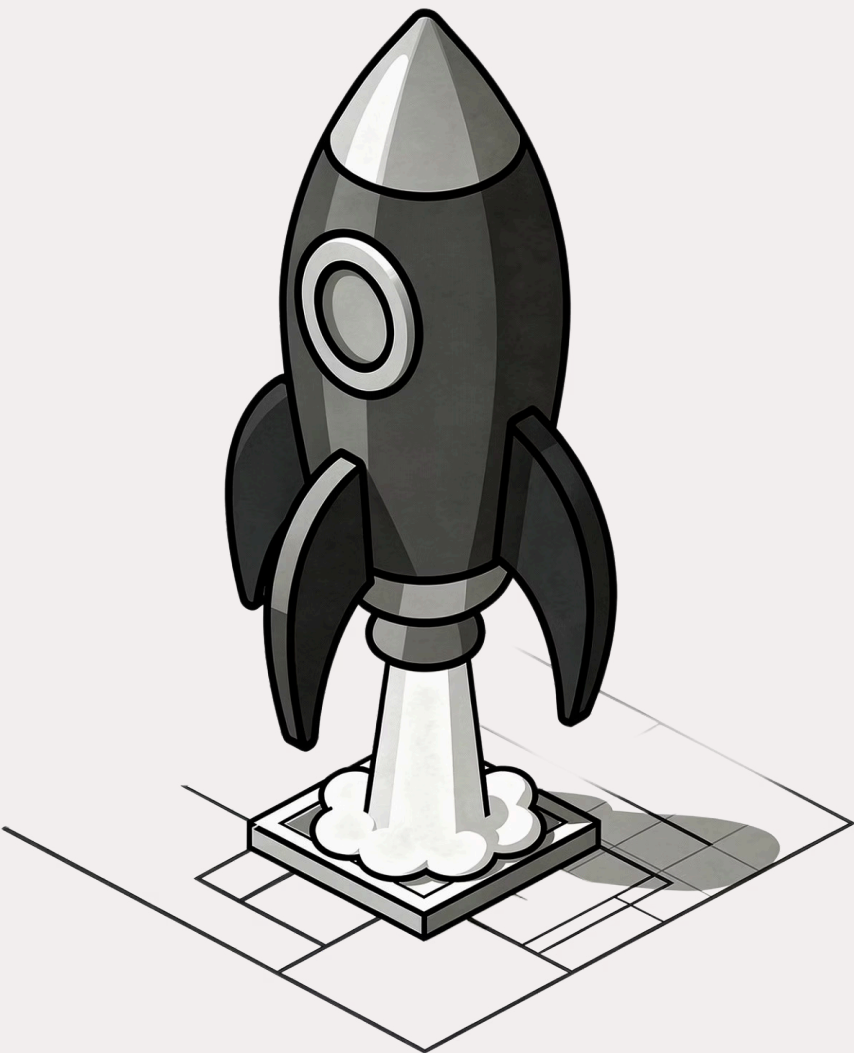
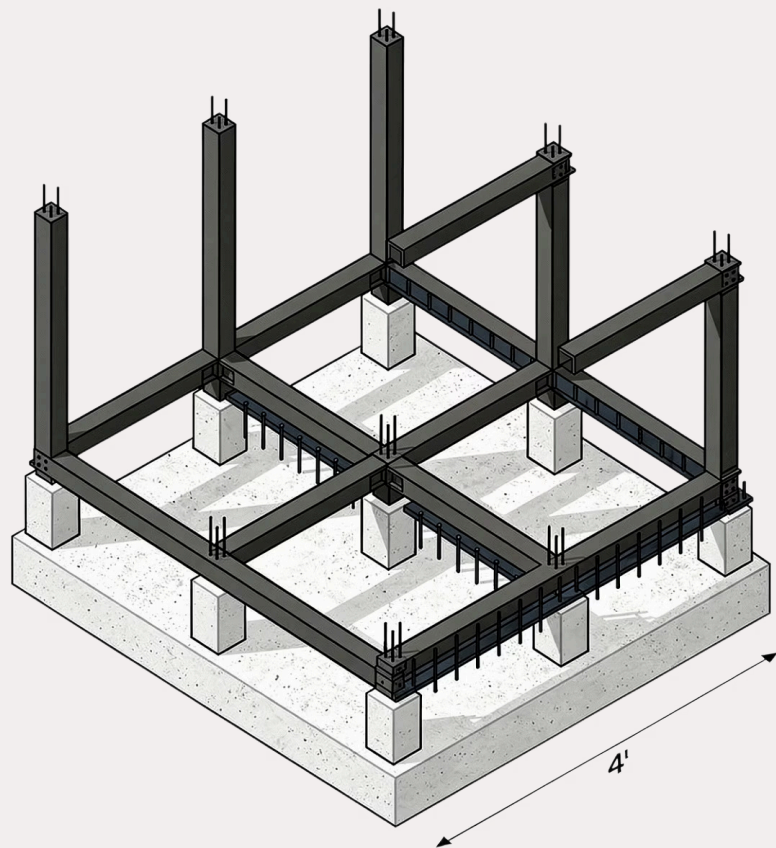




The Astronomer Deployment Lifecycle: From Code to Production

A comprehensive guide to managing Dev, Stage, and Production environments — covering deployment workflows, Docker image management, environment variables, and connection configuration on Astro.



CHAPTER 1

The Foundation: Environments

Before a single DAG reaches your users, it travels through a carefully structured set of environments. Understanding how these environments are organized — and why — is the cornerstone of a reliable, production-grade Astronomer deployment practice.

Dev, Stage, Production: A Three-Act Play



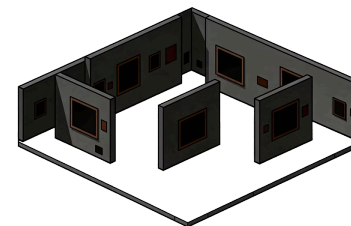
Development

Where ideas are born and code is written. The Dev environment is optimized for **rapid iteration and experimentation** — developers can push frequently, break things safely, and iterate without fear.



Staging

A **near-replica of production** used for rigorous testing and validation. Stage catches configuration drift, integration issues, and edge cases before they reach live workloads.



Production

The **live environment** where your Airflow DAGs serve business-critical functions. Changes here are deliberate, tested, and deployed with full confidence from upstream validation.

Why Multiple Environments?

Running parallel environments is not overhead — it's a strategic investment in reliability and team velocity.

Mitigate Risk

Catch bugs, schema mismatches, and dependency conflicts in lower environments **before they impact real users**. A failure in Dev costs minutes; a failure in Production can cost hours of downtime and business trust.

Enable Collaboration

Teams can develop, test, and validate features **independently and in parallel** without stepping on each other's work. Feature branches map cleanly to isolated deployment environments.

Ensure Quality

Rigorous, multi-stage testing produces **more stable and reliable deployments**. Each promotion gate — Dev → Stage → Prod — acts as a quality checkpoint, building confidence incrementally.

Environments as Creative Stages

Think of your deployment lifecycle like a creative process: **Development** is the sketch — messy, exploratory, full of possibility. **Staging** is the refinement — deliberate, precise, approaching perfection. **Production** is the gallery exhibit — polished, presented, and ready for the world.

1

Dev

Sketch & Experiment

2

Stage

Refine & Validate

3

Production

Exhibit & Deliver



CHAPTER 2

The Workflow: From Commit to Cloud

A well-designed CI/CD pipeline is the backbone of a mature Astronomer deployment practice. This chapter walks through how code travels from a developer's local machine all the way to a live Astro Deployment — automatically, reliably, and at scale.



The CI/CD Pipeline: Automating the Journey

What is CI/CD?

CI/CD stands for **Continuous Integration and Continuous Delivery** — a set of practices that automate the testing and deployment of software changes, eliminating slow manual handoffs.

- Code committed → tests run automatically
- Tests pass → deployment triggers automatically
- Each environment gate ensures only validated code advances

Key Benefits for Airflow Teams

- **Faster Development**
Remove manual deploy steps and ship DAG changes in minutes, not hours.
- **Increased Reliability**
Automated test gates catch regressions before they reach Production.
- **Reduced Human Error**
Scripted pipelines are consistent and repeatable — no forgotten steps.

Astronomer's CI/CD Strategy

Astronomer is designed to integrate seamlessly with modern CI/CD tooling. Here's how a typical Astro-native pipeline is structured:

Version Control

Host your Astro project source code in **Git** — GitHub, GitLab, or Bitbucket. Branch strategies (e.g., feature branches) map directly to deployment environments.

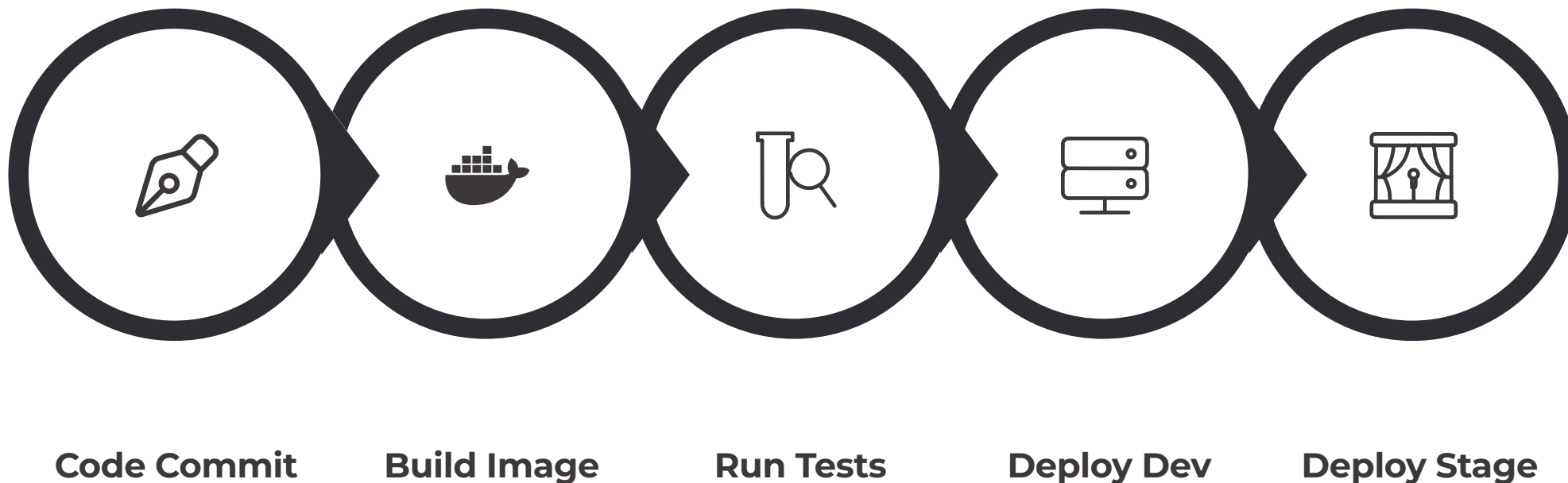
Automated Deploys

Use **GitHub Actions, GitLab CI, or CircleCI** workflows to automatically promote code through environments on merge or tag events — no manual ``astro deploy`` required.

API Tokens

Leverage **Deployment API tokens** for secure, non-interactive authentication in automated pipelines. Tokens are scoped per deployment to enforce least-privilege access.

The CI/CD Flow: End to End

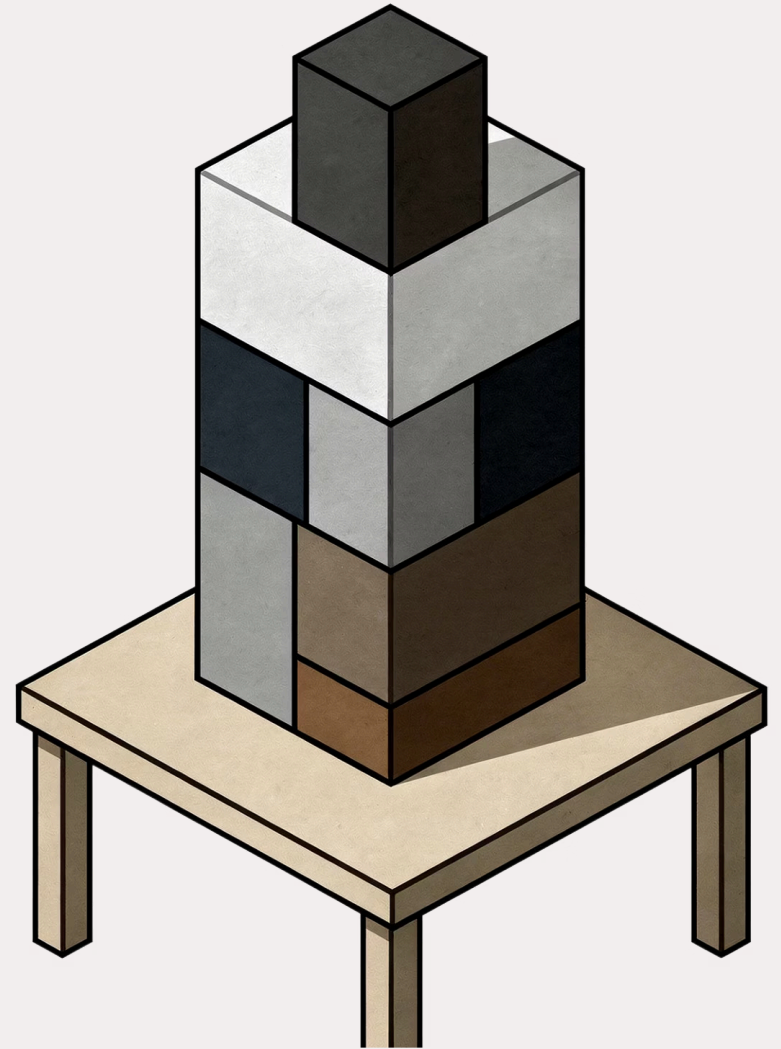


Every stage in this pipeline acts as a quality gate. Code that fails tests never advances downstream — ensuring only stable, validated changes reach Production. Astro's API tokens and deployment automation handle each promotion step without manual intervention.

CHAPTER 3

The Building Blocks: Docker Images and Variables

Beneath every Astro deployment is a Docker image and a set of environment variables. Understanding how to manage these artifacts through their full lifecycle — from build to registry to runtime — is essential for reproducible, environment-safe deployments.



Docker Image Registry Lifecycle

01

Build

Docker images encapsulate your **Airflow environment, providers, and custom dependencies**. The Astro CLI builds images locally or in CI from your project's `Dockerfile` and `requirements.txt`.

02

Version & Tag

Each image should be **versioned and tagged** — typically with a Git SHA or semantic version — so deployments are reproducible and rollbacks are trivial.

03

Push to Registry

Images are pushed to Astro's managed registry (or a private registry). Astro's image deploy templates automate this **build-push-deploy flow** within your CI pipeline.

04

Deploy

The Astro API pulls the versioned image and applies it to the target Deployment, triggering a **rolling restart** with zero DAG downtime.



 Astro's image deploy templates handle the full build-push-deploy cycle automatically inside your CI workflow — just configure once and let automation take over.

Environment Variables: Dynamic Configuration

Environment variables are the mechanism by which a single codebase adapts its behavior across Dev, Stage, and Production — without any code changes.



Purpose & Scope

Store **configuration values that vary by environment** — database URLs, API endpoints, feature flags, and resource limits. The same DAG reads different values in each environment automatically.



Security First

Never hardcode secrets in source code. Use **Astro's secret environment variables** for sensitive values like API keys and credentials — these are encrypted at rest and never exposed in the UI.



Astro Integration

Preview Deployments can access **secret Airflow variables via GitHub Secrets**, passed securely at deploy time. This enables full parity between ephemeral and permanent environments.

Versioned Images: Reproducibility by Design

Each environment runs a **precisely versioned Docker image**, giving you full control over what's running where. When Dev, Stage, and Prod each reference a specific image tag, rollbacks become one-line changes — and debugging becomes dramatically simpler because you know *exactly* what code is deployed.

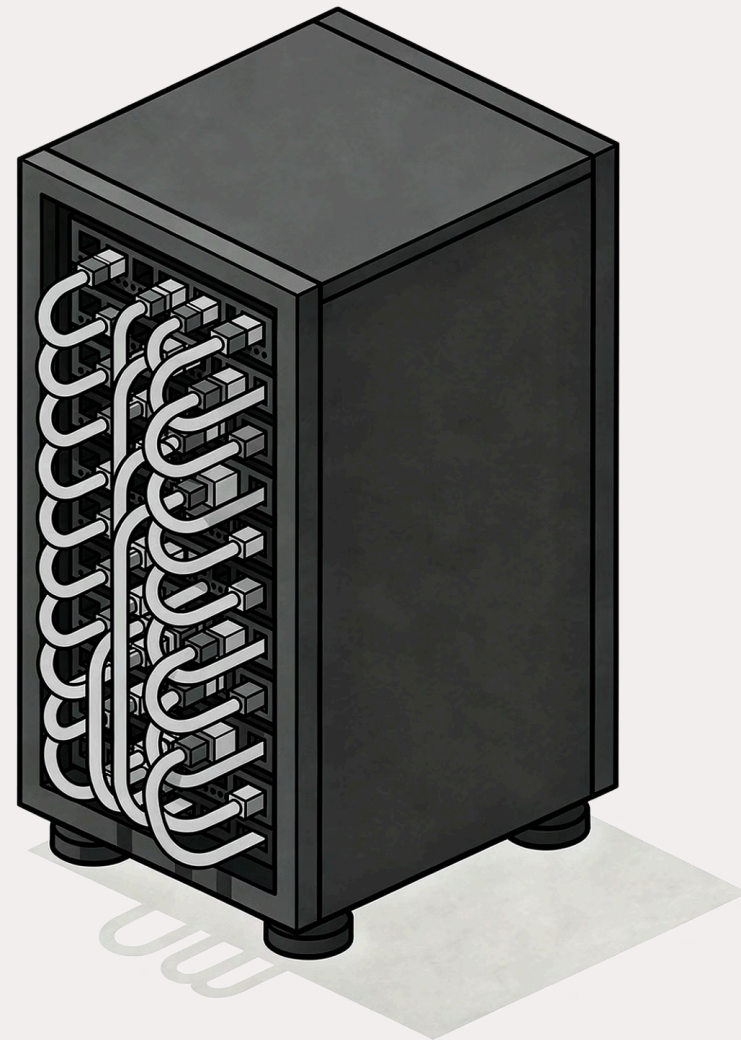
Dev image:feature-abc-sha123	Stage image:release-1.4.0-rc2
Production image:release-1.4.0	



CHAPTER 4

Connecting the Dots: Configuration and Best Practices

The final mile of a robust deployment lifecycle is connection management — ensuring each environment talks to the *right* external services with the *right* credentials, without leaking production access into lower environments.



Connection Configuration: Bridging Environments

What are Airflow Connections?

Airflow Connections define **how your DAGs interact with external systems** — databases, data warehouses, REST APIs, cloud storage, and more. Each connection stores a host, port, credentials, and optional extra config.

- Best Practice: Use the **same Connection ID** across all environments (e.g., `my_postgres_db`), but override the underlying connection details per environment. Your DAG code never changes — only the target system does.

Environment-Specific Details

Dev

Points to a local or sandboxed database. Low-stakes, disposable data.

Stage

Points to a staging replica of production systems with anonymized or synthetic data.

Production

Points to live systems with full production credentials, tightly access-controlled.

Managing Connections Across Branches

As teams scale, managing connections environment-by-environment becomes complex. Astronomer's tooling centralizes and automates this.



Branch-Based Deployments

Automatically associate specific Airflow connections with a deployment based on the **Git branch context**. Feature branches inherit Dev connections; release branches inherit Stage connections.



Astro Environment Manager

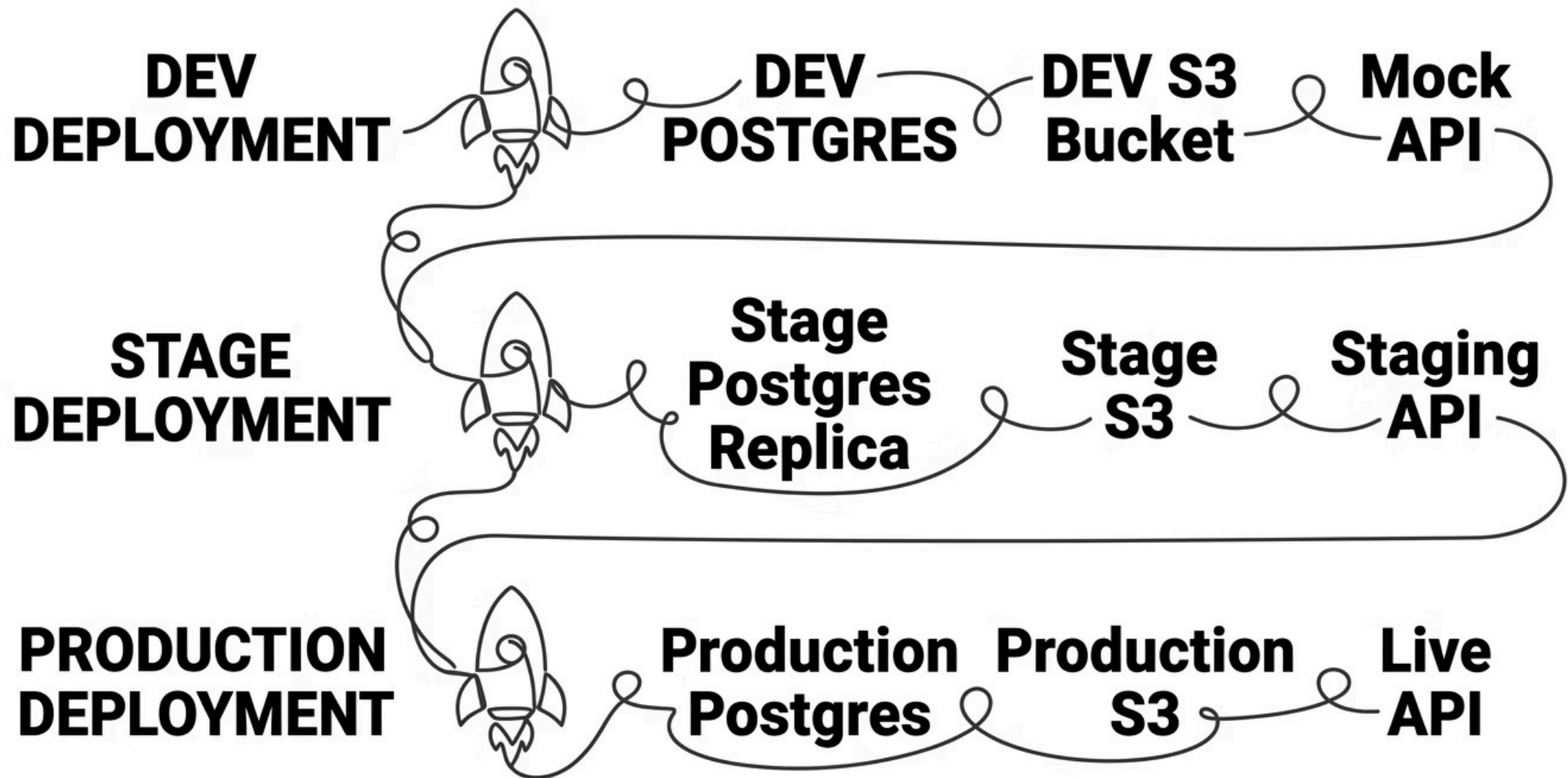
Configure connections **centrally in the Astro UI** and make them available to one or many Deployments simultaneously. Eliminates per-deployment manual configuration and reduces drift.



Override Capabilities

Developers can **override preset connections** in their personal Dev deployments for troubleshooting or custom testing — without affecting shared Stage or Production environments.

Connections Across Deployments



Same connection IDs. Different targets. This pattern ensures your DAG code is fully portable across environments — the infrastructure adapts, not the logic.

Choosing Your Deploy Strategy

Permanent Dev Deployments

- Simpler initial setup — create once, reuse always
- **Hibernation mode** reduces cost when idle
- Well-suited for smaller teams or single-developer workflows
- Shared Dev environment encourages coordination
- Less CI/CD complexity — a single target branch



Best for: Teams prioritizing simplicity and cost predictability.

Preview (Ephemeral) Deployments

- Spun up automatically per PR or feature branch
- **Full isolation** — each developer works in their own environment
- Destroyed on merge, keeping infrastructure lean
- Requires more sophisticated CI/CD pipeline setup
- Enables parallel development at scale



Best for: Larger teams with mature CI/CD practices.

The Power of Automation

Manual deployments are a liability — slow, error-prone, and invisible. Full automation transforms your deployment practice from a bottleneck into a competitive advantage.



Automate astro deploy

Remove the human from routine deploys entirely. Triggered automatically on merge to `main`, release tags, or PR creation — every time, the same way, with no forgotten flags or environment mix-ups.



Full Visibility

Astro's **GitHub integration** surfaces Git metadata — branch, commit SHA, author, message — directly in the Deployment UI. You always know exactly what's running and who shipped it.



Best Practices

Write **descriptive commit messages**, enforce CI checks on DAG syntax before merging, and maintain a strict branch-to-environment mapping. These habits compound into dramatically fewer production incidents.

The Future of Deployment: Reliable, Scalable, and Secure

A mature Astronomer deployment lifecycle is not a one-time setup — it's a living practice that grows with your team and your data platform.

Embrace CI/CD

Automate every promotion gate — from commit to Production — for faster, safer, and fully auditable Airflow deployments on Astro.

Leverage Environment Config

Environment-specific variables, connections, and Docker images make your pipelines **robust, portable, and resilient** to infrastructure changes.

Your Code to Production

From the first git commit to a live DAG serving business-critical workflows — **automated, reliable, and repeatable** every time.

From code to production, automated.

